Regressão logística no scikit-learn, validação cruzada e regularização

Aula 23 · Ciência de Dados Aplicada ao Direito II

Julio Trecenti

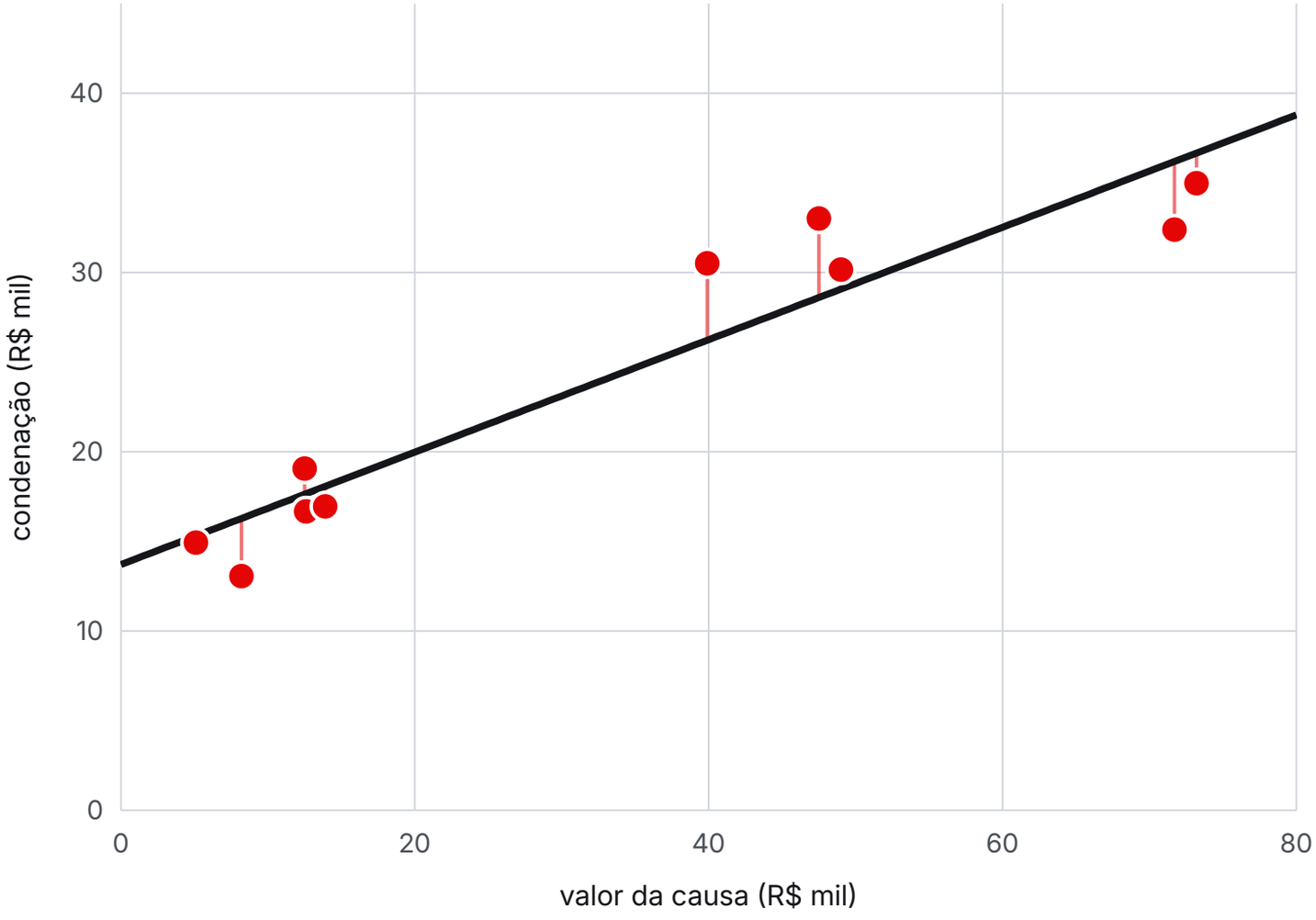
03/11/2026

Objetivos de aprendizagem

Ao final da aula, você será capaz de:

1. **reconhecer** sobreajuste e subajuste comparando o erro no treino e em casos novos;
2. **explicar** por que o teste fica guardado até a avaliação final;
3. **explicar** como o `C` controla a complexidade da regressão logística;
4. **diferenciar** parâmetro de hiperparâmetro;
5. **usar** a validação cruzada para escolher o hiperparâmetro, e a AUC para medir o modelo;
6. **ajustar** uma regressão logística no scikit-learn.

Sobreajuste e subajuste



complexidade (grau do polinômio): **1**

adicionar caso novo | recomeçar

erro médio no treino

2,2 R$ mil

erro médio nos casos novos (0 de 12)

—

COMO USAR

10 processos de treino (vermelho). Arraste a complexidade; depois, adicione **casos novos**, um de cada vez.

Exercício 4 da última aula: o modelo com o juiz

O escritório ajustou a regressão logística com a prova documental, o valor pedido e o **juiz**, em 220 processos de 80 juízes. A probabilidade que o modelo dá a um processo **novo**, com prova e pedido de R$ 10 mil:

juiz	processos no histórico	procedentes	probabilidade no processo novo
J11	3	0	**0%**
J69	2	0	**0%**
J76	1	0	**0%**
J68	3	3	**100%**
J03	2	2	**100%**
J61	5	5	**100%**

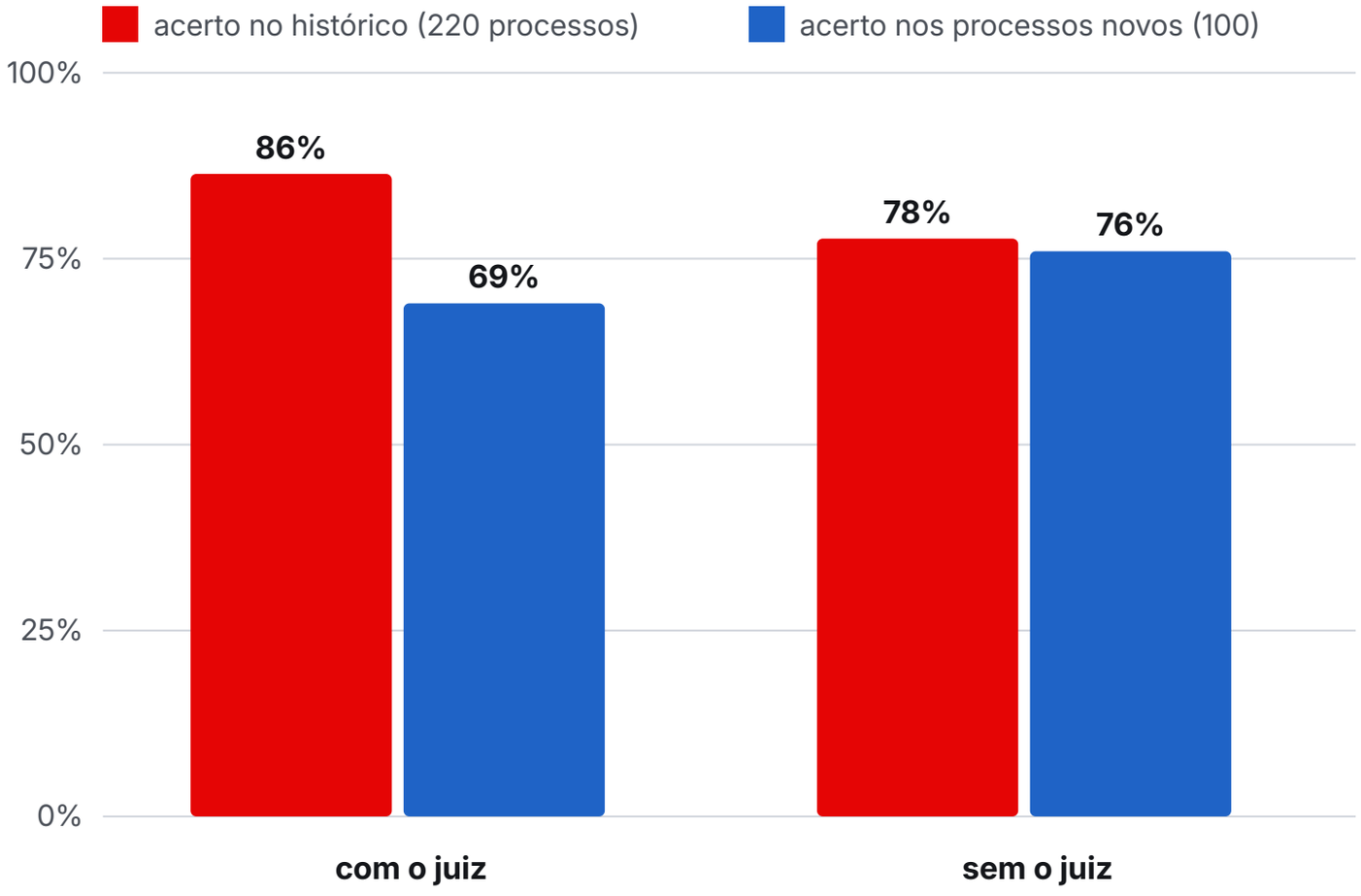
O QUE O MODELO APRENDEU

- Menos de **3 processos** por juiz, em média.
- Um único processo improcedente, e o J76 vira **0%** para sempre.
- Acerto **no histórico**: **86%**.

PERGUNTA

Você confiaria nessas probabilidades para os próximos processos?

Exercício 4: e nos processos novos?



A BASE NOVA

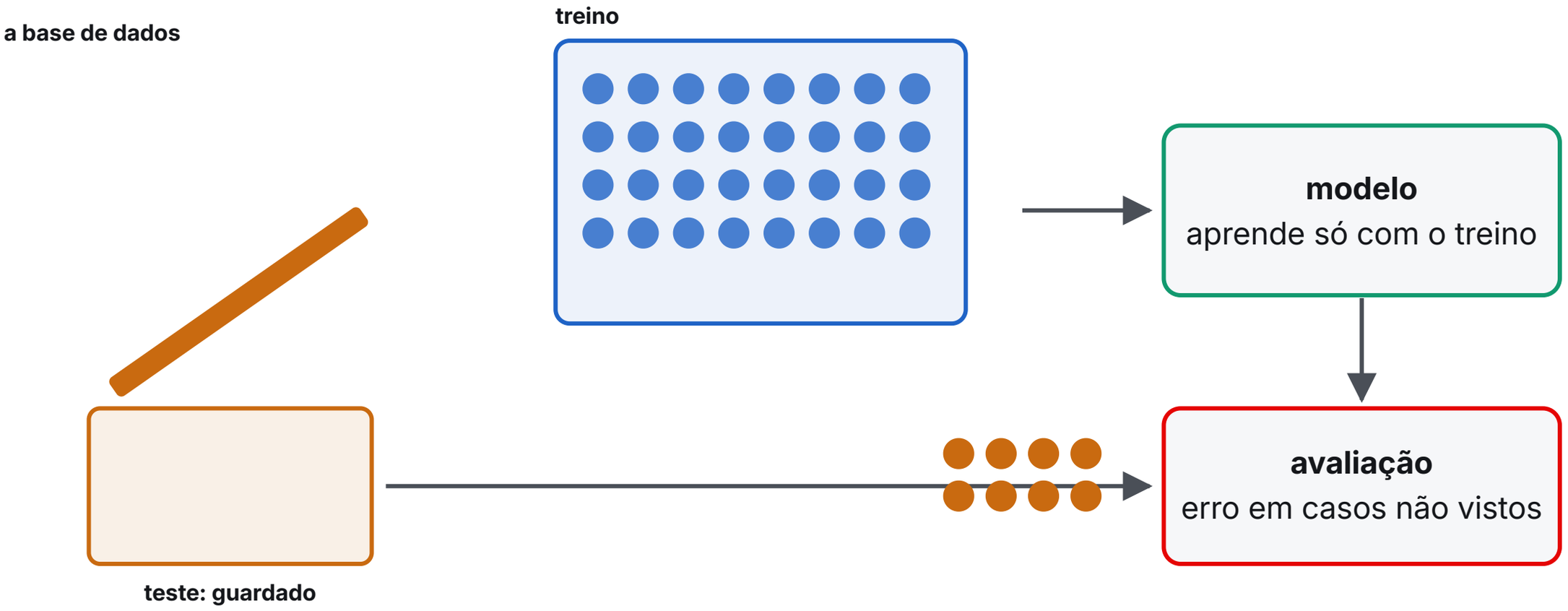
100 processos que o modelo não viu, dos mesmos juízes.

- Com o juiz: 86% no histórico, **69%** nos novos.
- Sem o juiz: 78% e **76%**.

SOBREAJUSTE

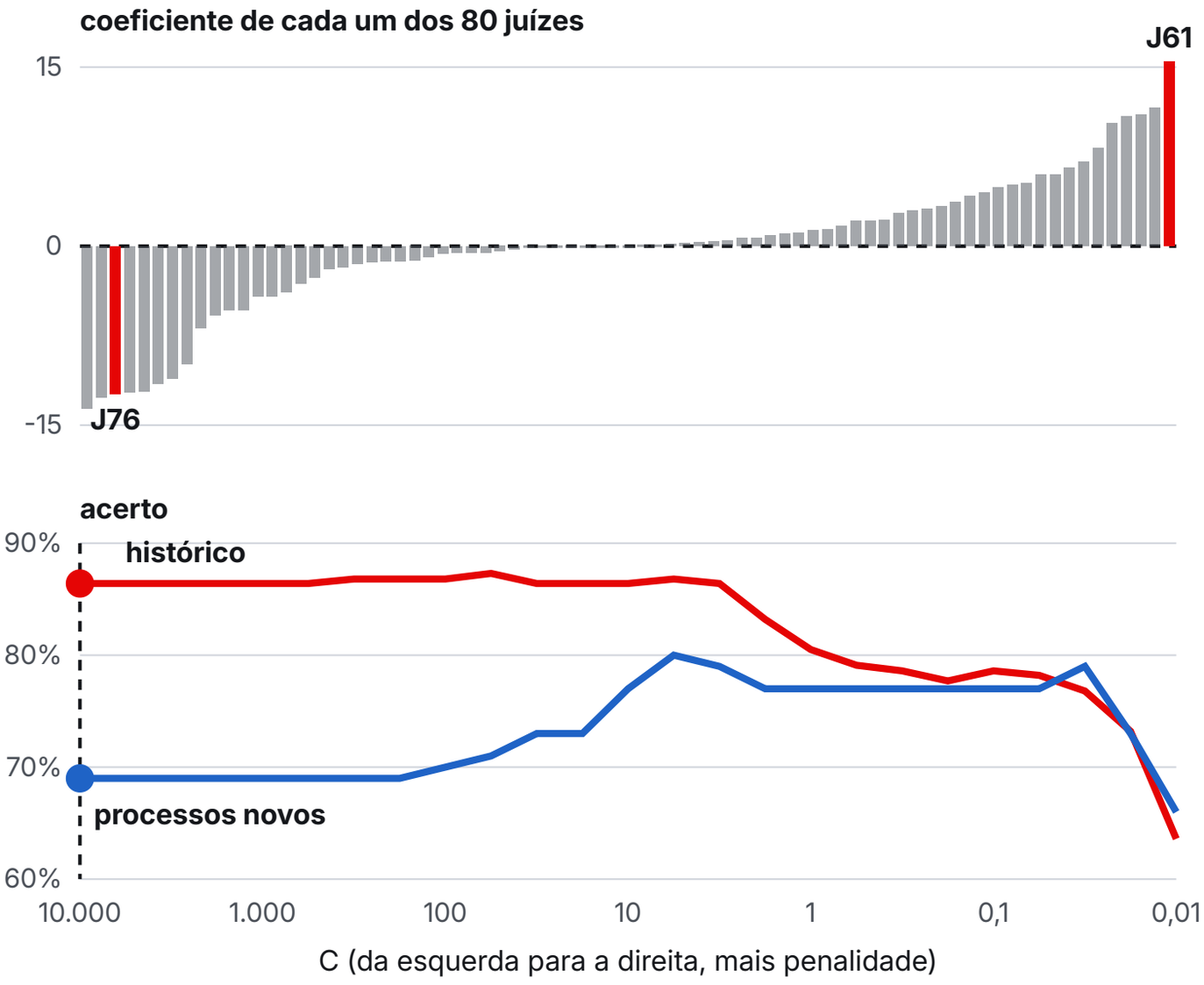
O modelo **decorou** o histórico, inclusive o acaso. Só os casos novos mostram isso.

Treino e teste: o teste fica guardado



No fim, a caixa abre **uma vez**: o erro no teste é a estimativa honesta do erro em casos novos. No exercício 4, os 100 processos novos foram o teste.

A complexidade da regressão logística: o C



penalidade: **C = 10.000**

```
modelo = LogisticRegression(C=10000)
modelo.fit(X_historico, y_historico)
```

acerto no histórico
86%

acerto nos novos
69%

caso novo no J76
0%

caso novo no J61
100%

O QUE O c FAZ

O ajuste **paga** por coeficiente grande, e o C diz quanto: C **menor, penalidade maior**. Os coeficientes dos juízes encolhem, e o J76 volta para perto da média.

Parâmetros e hiperparâmetros

PARÂMETRO: SAI DOS DADOS, NO AJUSTE

- Os **coeficientes** da regressão: o efeito da prova documental, do valor pedido, de cada juiz.
- O computador calcula, no `fit`, sem a nossa interferência.
- No polinômio de grau 8, os 9 coeficientes da curva.

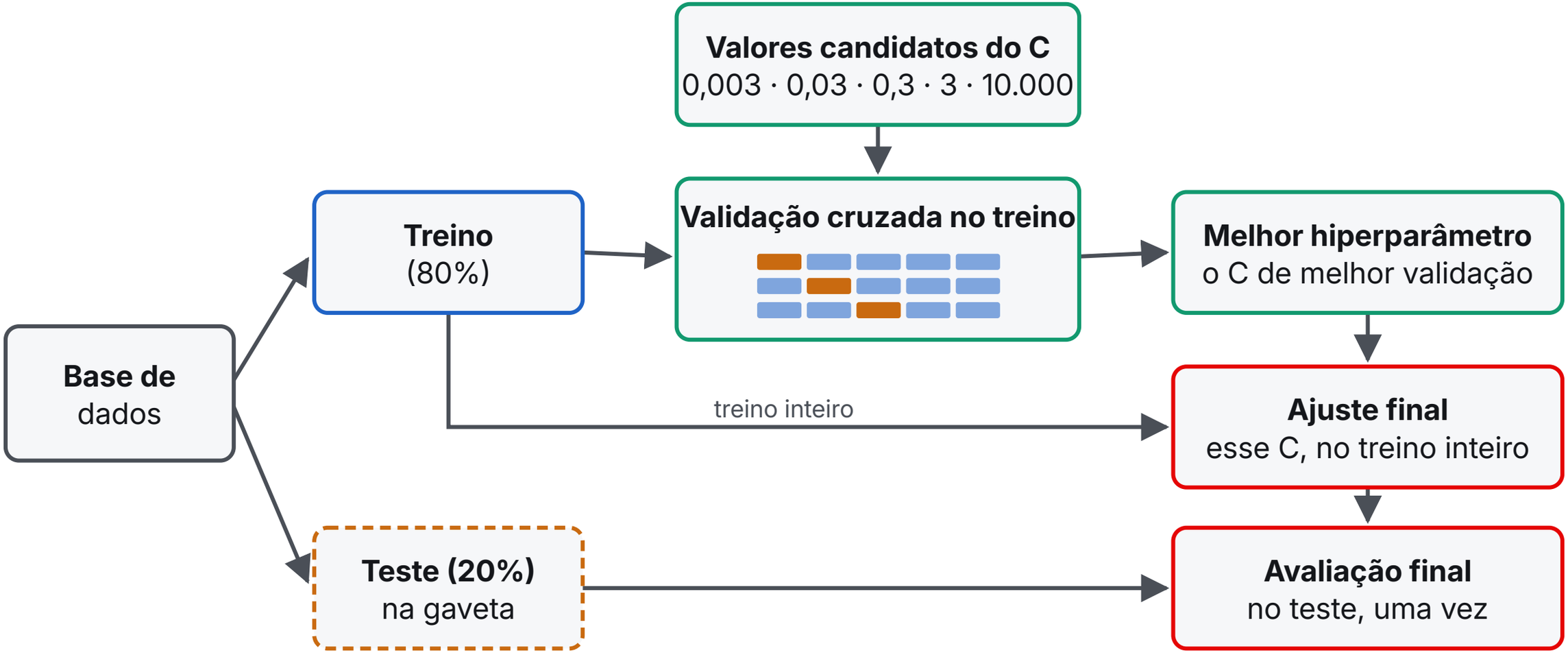
HIPERPARÂMETRO: FIXADO ANTES DO AJUSTE

- Um botão do **método**, que controla a **complexidade** do modelo.
- O grau do polinômio; na regressão logística, o `C`; na aula 24, a profundidade da árvore e o número de árvores.
- O treino sempre prefere o mais complexo, e o teste está na caixa: **como escolher?**

NA PRÁTICA

Para cada valor do hiperparâmetro, o método ajusta os parâmetros. Falta um jeito de comparar esses modelos sem abrir a caixa do teste.

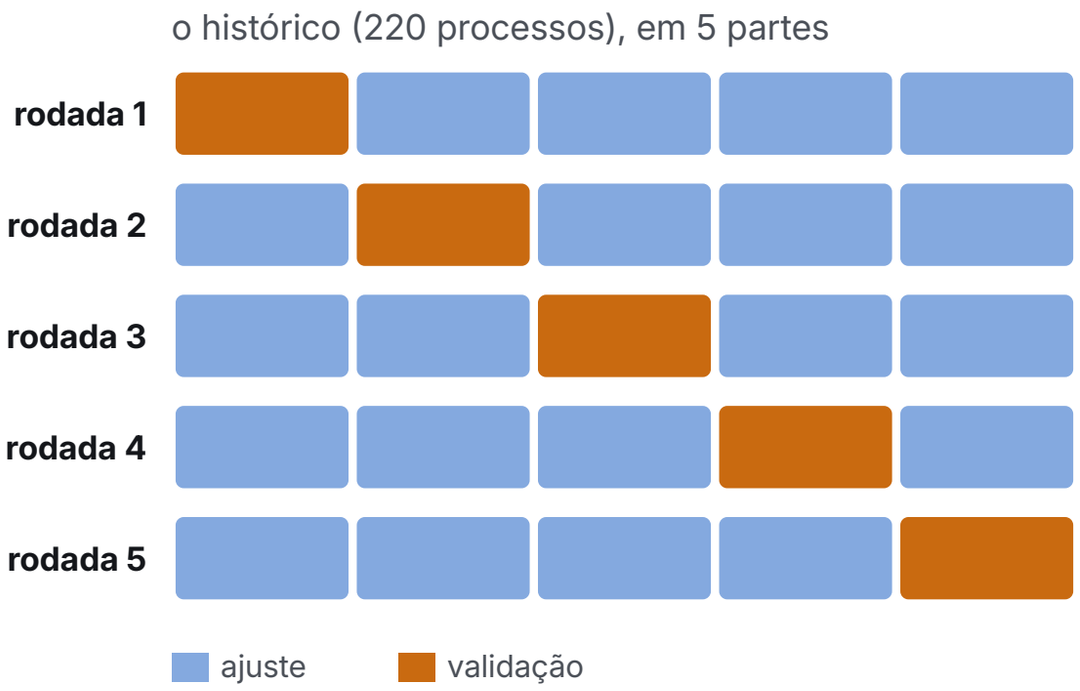
O fluxo de um modelo de machine learning

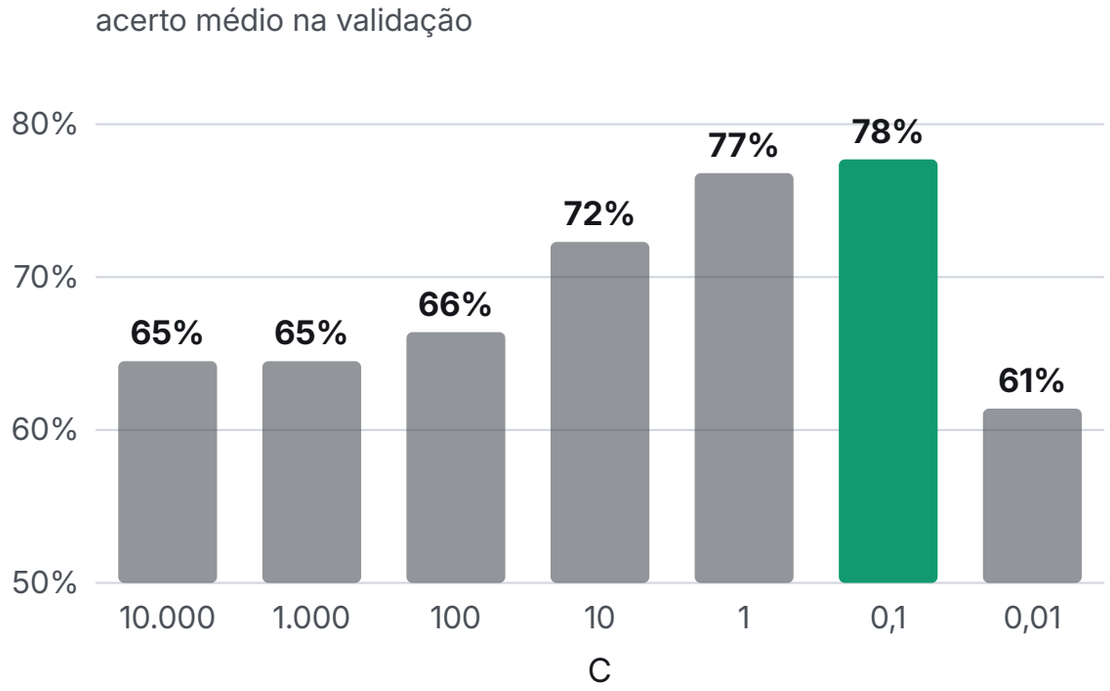


POR QUE O TESTE FICA NA GAVETA

Se o teste for usado para escolher o `C`, ele vira parte do treino, e a AUC relatada fica **otimista**. Adaptado do diagrama do scikit-learn sobre a busca de hiperparâmetros.

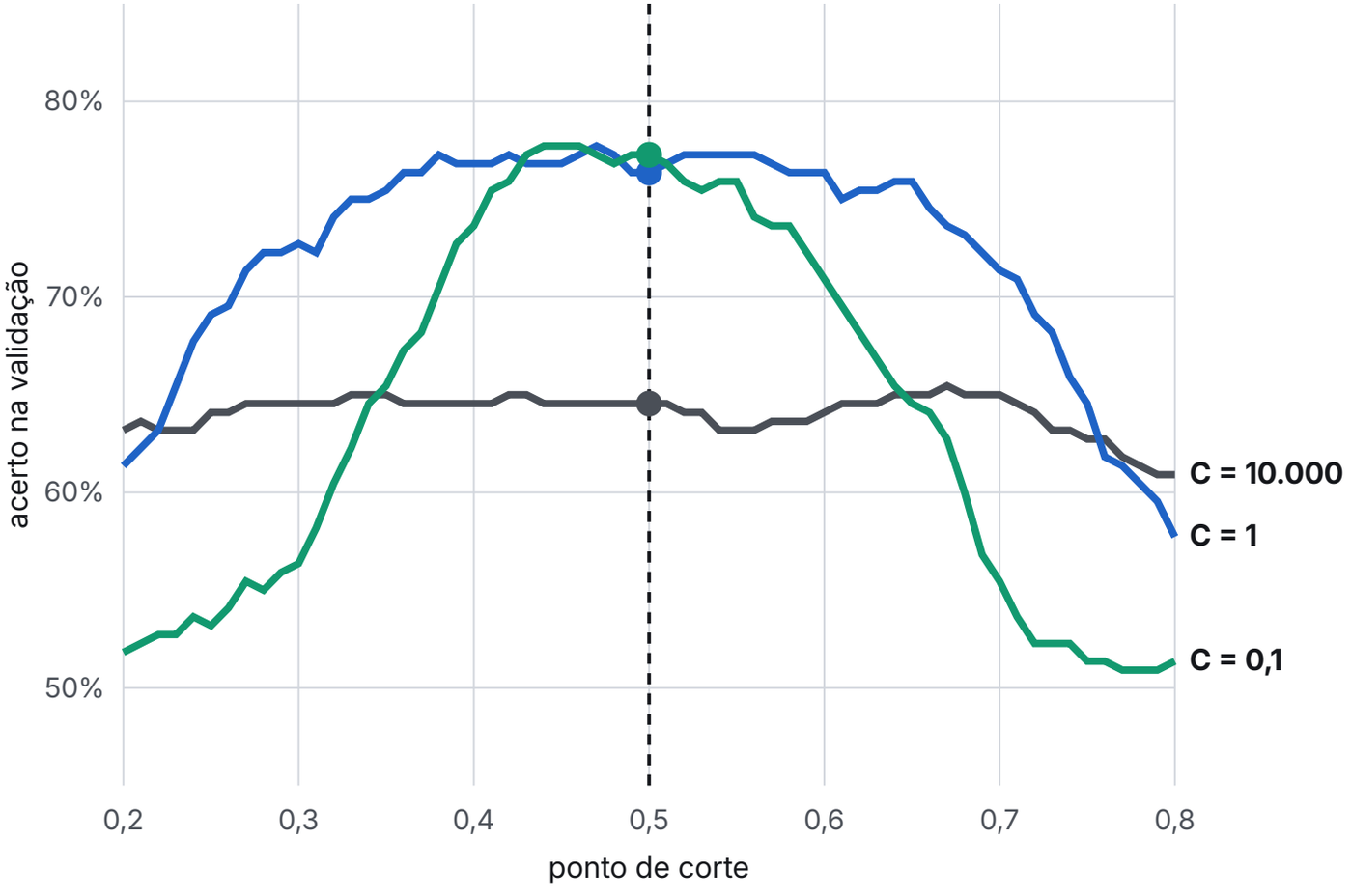
Validação cruzada: escolher o C sem abrir o teste





O C com maior acerto médio na validação é o escolhido: **0,1**. Os 100 processos novos não foram usados para escolher.

O acerto depende do ponto de corte



ponto de corte: **0,50**

C = 10.000	C = 1	C = 0,1
65%	**76%**	**77%**

UM PROBLEMA NA VALIDAÇÃO CRUZADA

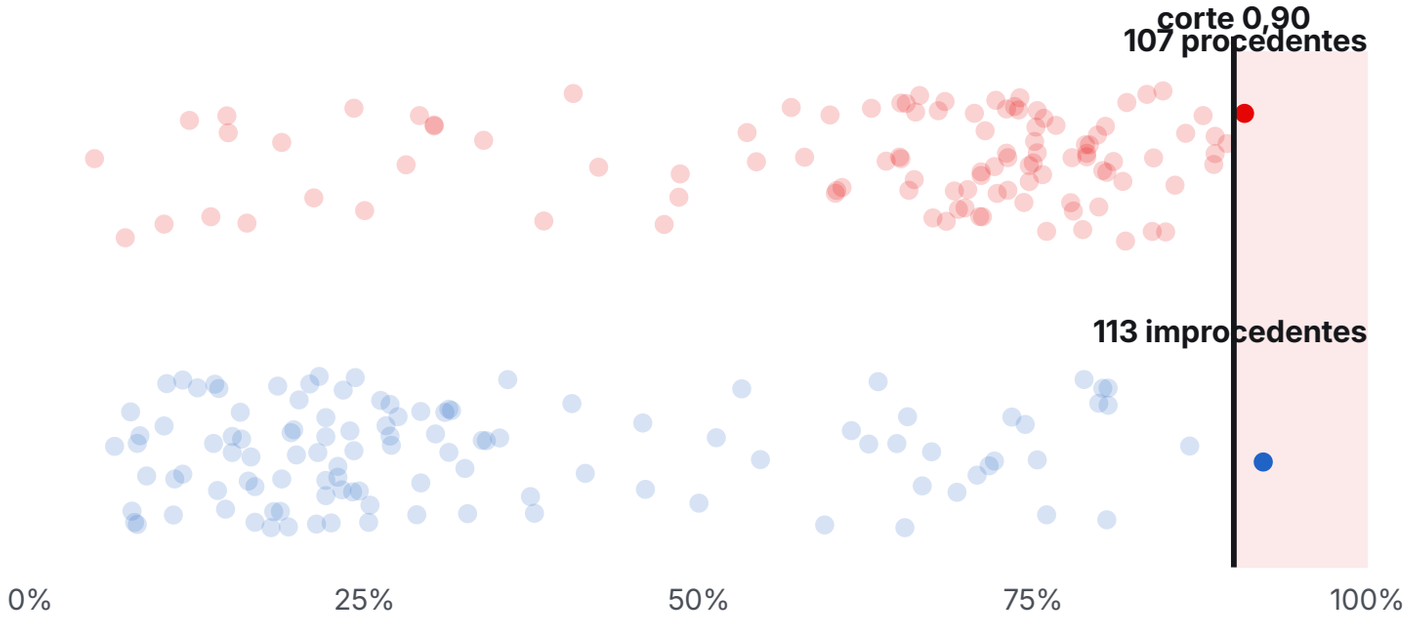
Para medir o acerto, é preciso escolher um **corte**. No corte 0,5, vence `C = 0,1`; no 0,3 ou no 0,7, vence `C = 1`. A escolha do `C` fica refém de um corte que nem decidimos. Precisamos de uma medida que olhe **todos os cortes**.

Relembrando: sensibilidade, especificidade e precisão

	marcado prob. ≥ 0,5	não marcado prob. < 0,5	total
procedentes	verdadeiro positivo **84**	falso negativo 23	107
improcedentes	falso positivo **29** → **×2**	verdadeiro negativo 84	113
	113	107	220

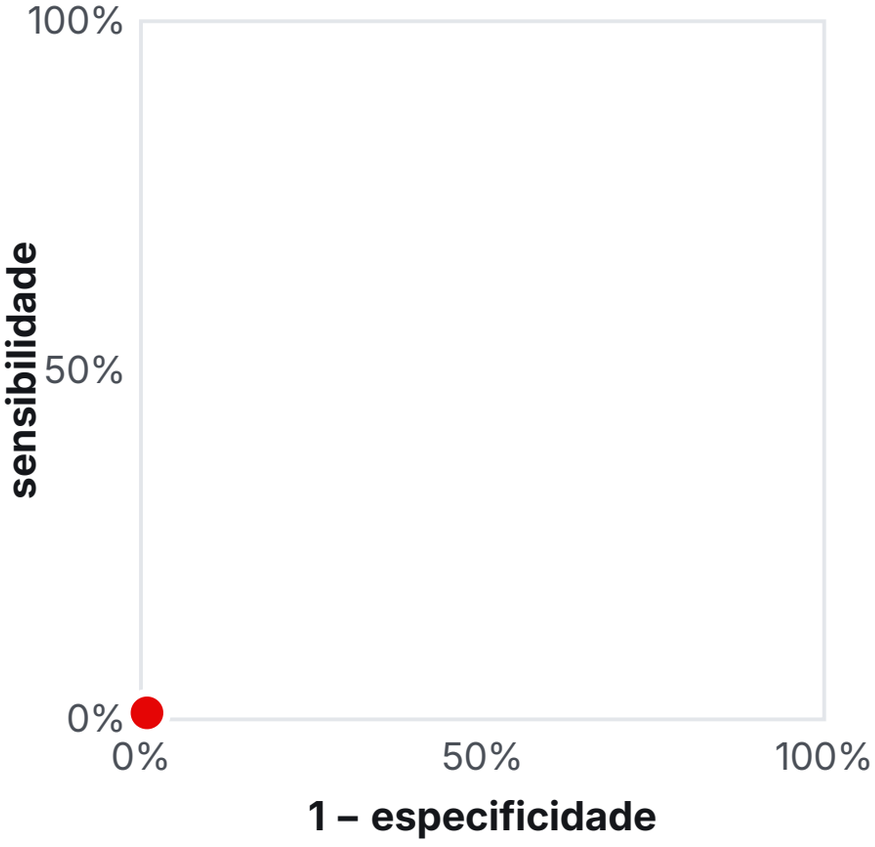
Por que não a precisão? Se a base tivesse o **dobro de improcedentes**, com o mesmo modelo, a precisão cairia para **84 / 142 = 59%**. A sensibilidade (**79%**) e a 1 – especificidade (**26%**) não mudam: cada uma olha um grupo só. Elas medem **o modelo**, e não a mistura da base. Por isso a curva ROC e a AUC são feitas com elas.

Sensibilidade e 1 − especificidade: os dois lados de um corte



ponto de corte: **0,90**

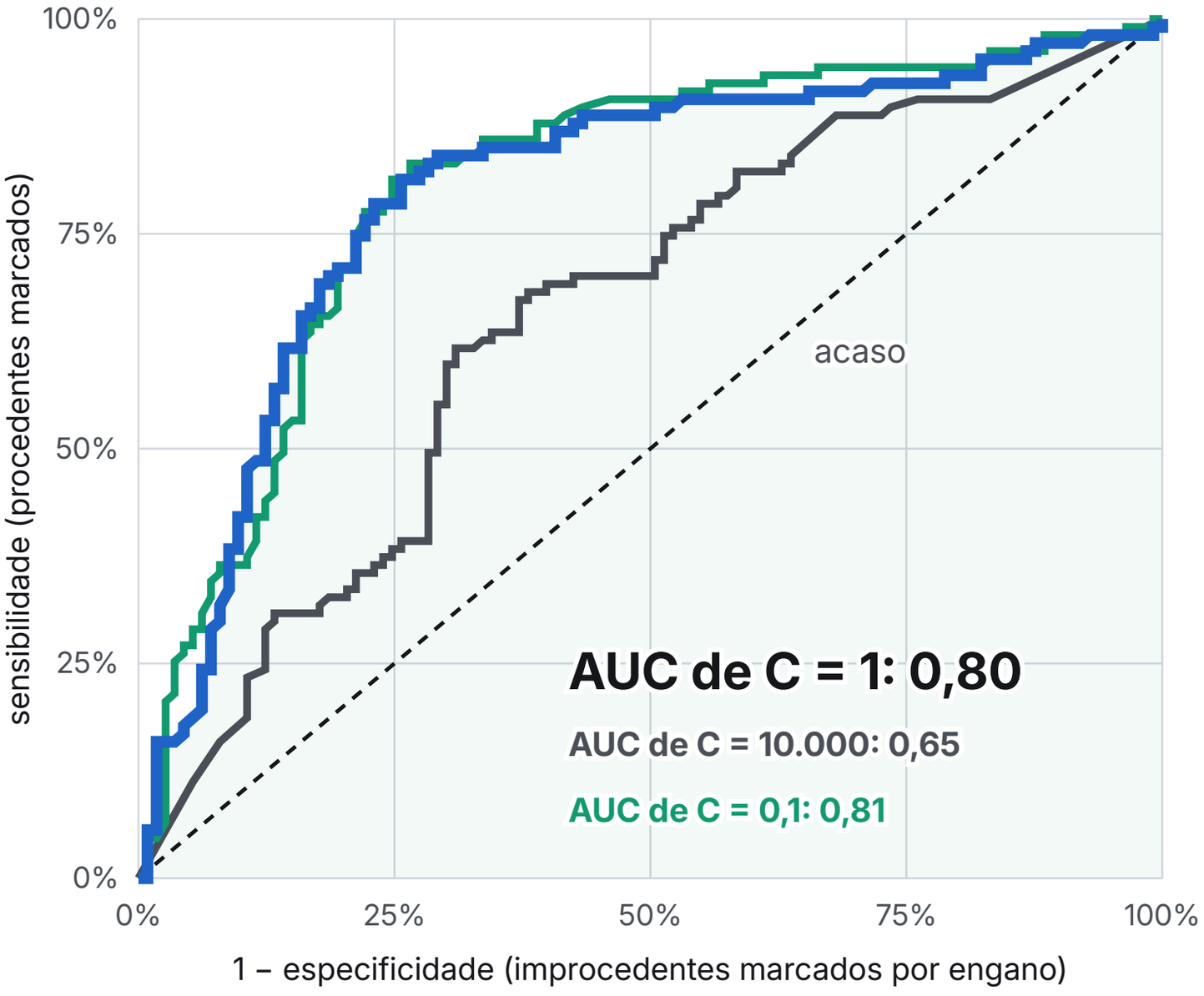
sensibilidade	1 − especificidade
1%	**1%**
1 de 107 procedentes	1 de 113 improcedentes



POR QUE ESSAS DUAS MEDIDAS

Cada uma é medida **dentro de um grupo**, então não depende de quantos procedentes há na base. Baixar o corte aumenta as duas: pega mais procedentes (**o ganho**) e marca mais improcedentes por engano (**o custo**).

A curva ROC: todos os cortes de uma vez



Agora dá para comparar os `C` sem escolher corte: **0,1** e **1** quase empatam, e **10.000** fica muito atrás. A validação cruzada do notebook usa a AUC.

AUC: uma medida que não depende do corte

COMO INTERPRETAR A AUC

Sorteie um processo **procedente** e um **improcedente**. A AUC é a probabilidade de o modelo dar ao procedente uma probabilidade maior que ao improcedente.

- **0,5:** o modelo ordena como uma moeda.
- **1:** ordena perfeitamente.
- `C = 0,1` **no juizado: 0,81.** Em 81% dos pares, o procedente fica na frente.

O QUE A AUC NÃO DIZ

- Não escolhe o corte: a mesma AUC serve para qualquer ponto da curva. O corte vem depois, pelo custo de cada erro (como no Projeto 06).
- No notebook, a validação cruzada mede cada `C` pela **AUC**. No **Projeto 08**, a nota de cada modelo é a AUC calculada no servidor.

Regressão logística no scikit-learn

```
# o preparo faz as dummies e padroniza
modelo = make_pipeline(preparo,
    LogisticRegression(C=0.3))

# validação cruzada
auc = cross_val_score(modelo, X_treino,
    y_treino, cv=5, scoring="roc_auc")
auc.mean()

# ajusta no treino inteiro
modelo.fit(X_treino, y_treino)

# teste, uma vez
prob = modelo.predict_proba(X_teste)[:, 1]
roc_auc_score(y_teste, prob)
```

DIFERENÇAS PARA O STATSMODELS

- `X` e `y` separados, sem fórmula.
- `make_pipeline` guarda o preparo junto com o modelo: as dummies são feitas **dentro** de cada rodada.
- `fit` ajusta e `predict_proba` dá as probabilidades.
- A penalidade já vem ligada: `C=1`, por padrão.
- Não há tabela com valor-p: o foco é **prever**.

Exercício no notebook: a base do Projeto 08

NOTEBOOK DA AULA 23 · INDIVIDUAL · PARA FAZER AGORA

Recursos criminais do TJSP, com outra pergunta: **houve voto divergente?** (2% dos acórdãos).

1. Separar treino e teste, e calcular o modelo trivial.
2. Ajustar a logística e comparar treino e teste.
3. Validação cruzada.
4. Regularização: escolher o `C`.
5. Comparar modelos e abrir a gaveta.

Abrir o notebook no Colab

POR QUE A AUC AQUI

Com 2% de eventos, o trivial acerta 98%, e a acurácia não diz nada. A **AUC** mede quanto o modelo ordena os casos, e é ela que vale no **Projeto 08**.

Quando terminar, guarde o notebook: na quinta ele continua, e o código serve de base para o projeto.

Resumo da aula e próximas aulas

PONTOS PRINCIPAIS DA AULA

- **Sobreajuste:** acerta o treino, erra os casos novos.
- O **teste** fica na caixa e abre uma vez, no fim.
- O `C` é o hiperparâmetro de complexidade da logística: `C` **menor, penalidade maior**.
- A **validação cruzada** escolhe o hiperparâmetro; a **AUC** mede quanto o modelo ordena bem os casos.

PRÓXIMAS AULAS

- **05/11:** árvores e florestas, e lançamento do **Projeto 08** (competição de modelos, nota pela AUC).
- **10/11: Projeto 07** no início da aula (provinha de 5 questões sobre as aulas 22 a 24); depois, interpretabilidade, justiça algorítmica e LGPD.